

DEEP LEARNING

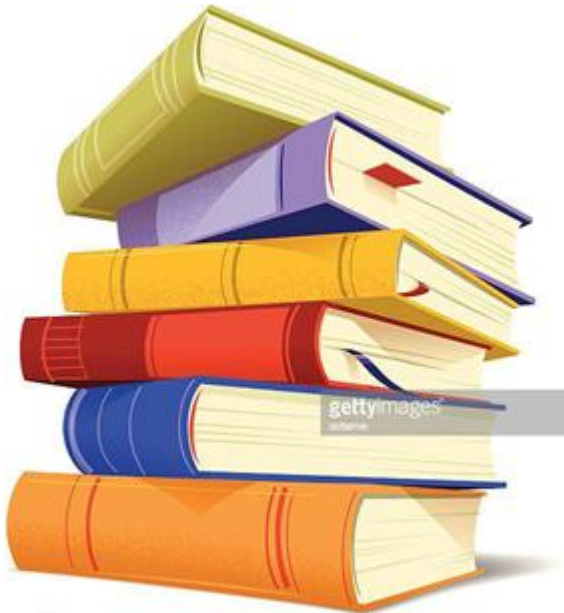
NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU

www.hoasen.edu.vn

Chương 7

Working with Keras: A deep dive
Làm việc chuyên sâu với Keras

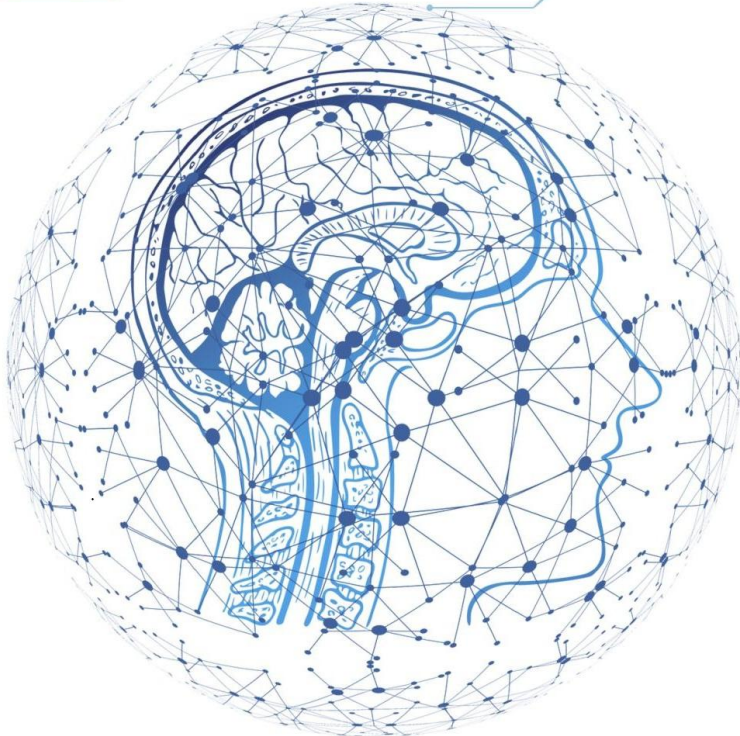




- **A spectrum of workflows**
Phổ các quy trình làm việc trong Keras
- **Different ways to build Keras models**
Các cách xây dựng mô hình Keras
- **Using built-in training and evaluation loops**
Dùng vòng lặp train/evaluate có sẵn
- **Writing your own training and evaluation loops**
Tự viết vòng lặp huấn luyện và đánh giá
- **Chapter summary**
Tổng kết các nguyên tắc quan trọng

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU



A spectrum of workflows
Phổ các quy trình làm việc

Why a Keras deep dive? Vì sao cần đào sâu Keras?

- Các bài toán thực tế thường **phức tạp hơn** ví dụ nhập môn
- Model có thể có nhiều input, nhiều output hoặc nhiều nhánh xử lý
- Quy trình train đôi khi cần metric, callback hoặc loss tùy biến
- Keras cho phép tăng dần mức kiểm soát mà vẫn giữ code rõ ràng
- **Mục tiêu:** biết khi nào dùng API có sẵn, khi nào cần tự viết

A spectrum of workflows

Phổ quy trình làm việc

- **Mức cao nhất:** Sequential + compile + fit cho model tuyến tính
- **Mức linh hoạt hơn:** Functional API cho đồ thị nhiều nhánh
- **Mức tùy biến kiến trúc:** subclass Model và viết call
- **Mức tùy biến huấn luyện:** custom train_step hoặc loop thủ công
- Càng xuống thấp càng linh hoạt, nhưng cũng càng dễ lỗi

Guiding principle

Nguyên tắc lựa chọn workflow

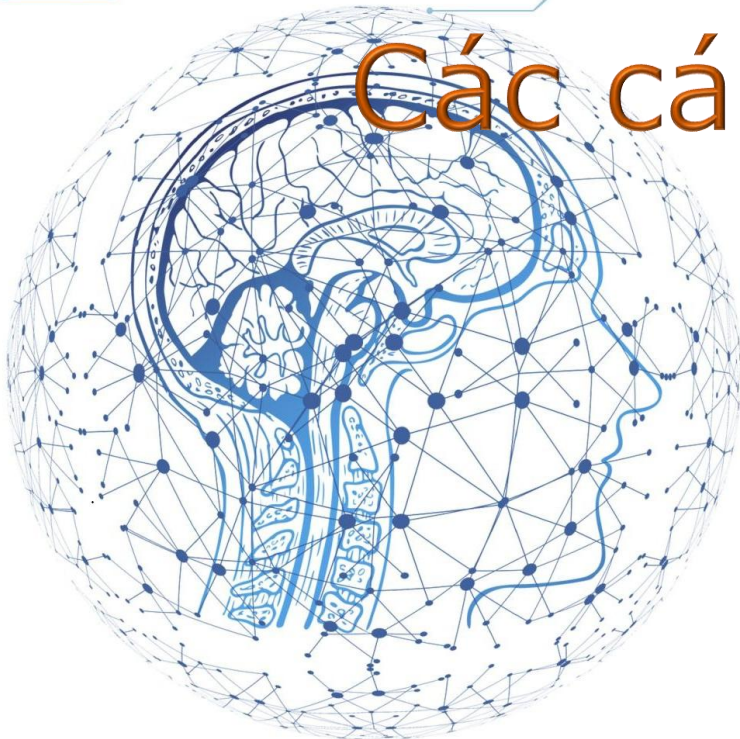
- **Nguyên tắc 1:** nếu Sequential đủ, không cần Functional API; nếu Functional API đủ, không cần subclassing.
- **Nguyên tắc 2:** càng dùng được nhiều tiện ích có sẵn thì code càng dễ đọc, dễ debug và dễ tái lập thí nghiệm.
- **Nguyên tắc 3:** xuống custom loop chỉ khi fit không thể diễn tả thuật toán train mong muốn.
- **Ví dụ:** model nhiều output nhưng train chuẩn vẫn dùng được Functional API + fit, không cần custom loop.
- **Kết luận:** chọn API giống chọn công cụ; công cụ tốt là công cụ vừa đủ cho bài toán.

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU

Different ways to build Keras models

Các cách xây dựng mô hình Keras



Three model-building APIs Ba API xây dựng model

- **Sequential:** xếp các layer thành một chuỗi tuyến tính
- **Functional API:** kết nối tensor thành đồ thị tính toán
- **Model subclassing:** viết model như một class Python tùy biến
- Model và layer đều có thể được gọi như hàm trên tensor
- Các cách này có thể **kết hợp** trong cùng một dự án
- **Ví dụ:** có thể xây encoder bằng Functional API, sau đó đưa encoder vào một subclassed model để viết logic train đặc biệt.

The Sequential model

Mô hình Sequential

- Là một **danh sách layer** chạy theo thứ tự từ đầu đến cuối
- Phù hợp với bài toán **một input, một output**
- Dễ viết, dễ đọc, dễ dùng cho baseline ban đầu
- Không phù hợp khi model có nhánh rẽ, nhiều input/output
- Nên dùng khi kiến trúc thật sự là một pipeline tuyến tính

Building and inspecting Sequential

Tạo và kiểm tra Sequential

- Có thể truyền danh sách layer ngay khi tạo model
- Cũng có thể thêm layer từng bước bằng phương thức add
- Layer tạo weight khi đã biết **input shape**
- Dùng **model.summary** để kiểm tra output shape và số tham số
- Đặt tên layer giúp đọc summary và debug thuận tiện hơn
- **Tại sao cần kiểm tra tham số:** số tham số quá lớn có thể dẫn đến overfitting hoặc chạy rất chậm.
- **Ví dụ:** nếu Dense layer nhận input 10 chiều và có 32 units, số weight là:

$$10 \times 32 \text{ cộng } 32 \text{ bias} = 352$$

Functional API: Mental model

Functional API: Mô hình đồ thị

- Bắt đầu bằng **keras.Input** để tạo **symbolic tensor**
- Gọi layer trên tensor để tạo tensor mới
- Đóng gói bằng **keras.Model** với inputs và outputs
- Keras biết rõ các node và cạnh trong đồ thị model
- Phù hợp với kiến trúc nhiều nhánh nhưng vẫn dễ quản lý

Functional API example

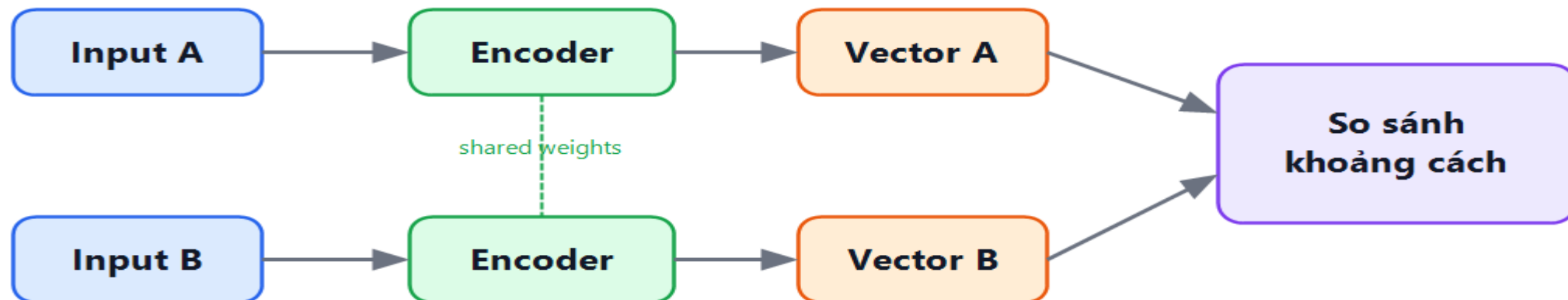
Ví dụ Functional API

- Một model có thể nhận nhiều loại dữ liệu cùng lúc
- Ví dụ ticket hỗ trợ: tiêu đề, nội dung chi tiết và tag
- Các nhánh xử lý riêng được ghép lại bằng **concatenate**
- Output có thể gồm mức ưu tiên và phòng ban phụ trách
- Mỗi output có thể dùng **loss và metric riêng**
- Functional API làm các bài toán đa input, đa output trở nên tự nhiên và rõ ràng.

Sharing and reusing layers

Chia sẻ và tái sử dụng layer

- Cùng một layer object có thể được gọi nhiều lần
- Các lần gọi đó **dùng chung weight**
- Hữu ích khi nhiều input cần cùng một bộ trích xuất đặc trưng
- Có thể tạo shared encoder hoặc siamese network
- Cần phân biệt shared layer với hai layer mới cùng cấu hình



Inspecting model connectivity

Kiểm tra kết nối của model

- Functional API lưu lại quan hệ giữa layer, input và output
- Có thể xem `model.layers` và `summary` để kiểm tra kiến trúc
- Có thể tạo model trung gian để lấy activation ở layer giữa
- Hữu ích cho debug, visualization và phân tích feature
- Đây là ưu điểm lớn so với model subclassing thuần

Subclassing the Model class

Kế thừa lớp Model

- Tạo class mới kế thừa keras.Model
- Khai báo các layer con trong `__init__`
- Viết luồng tính toán trong phương thức `call`
- Phù hợp với logic động, điều kiện hoặc vòng lặp Python
- Đổi lại: ít thông tin graph tự động hơn Functional API

Mixing components Kết hợp các thành phần

- Layer, model và custom block đều có thể được lắp ghép
- Một model nhỏ có thể trở thành block trong model lớn
- Có thể dùng custom layer trong Sequential hoặc Functional API
- Có thể dùng Functional model bên trong subclassed model
- Tư duy quan trọng: xây model lớn từ các khối nhỏ rõ nghĩa

Use the right tool for the job Dùng đúng công cụ cho bài toán

- Sequential: khi model là chuỗi layer đơn giản
- Functional API: lựa chọn mặc định cho model nhiều nhánh
- Subclassing: khi kiến trúc cần logic Python tùy biến
- Custom loop: khi thuật toán train cần kiểm soát sâu
- **Ưu tiên đơn giản, chỉ phức tạp hóa khi có lý do**

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU



Using built-in training
and evaluation loops
Dùng vòng lặp huấn luyện
và đánh giá có sẵn

Standard training workflow

Quy trình huấn luyện chuẩn

- **compile()** cấu hình optimizer, loss và metrics
- **fit()** huấn luyện model qua nhiều epoch và batch
- **evaluate()** đo hiệu năng trên validation hoặc test set
- **predict()** tạo output dự đoán cho dữ liệu mới
- **History** là object quan trọng lưu lại loss và metric theo từng epoch để vẽ learning curve và phân tích overfitting

Why built-in loops matter Vì sao nên dùng vòng lặp có sẵn?

- **Ít code hơn:** người học tập trung vào model và dữ liệu thay vì viết lại vòng lặp chuẩn.
- **Ít lỗi hơn:** Keras xử lý đúng thứ tự reset metric, validation, callback và logging.
- **Tích hợp hệ sinh thái:** callbacks như EarlyStopping, ModelCheckpoint, TensorBoard hoạt động trực tiếp với fit.
- **Dữ liệu đa dạng:** fit có thể nhận NumPy arrays, tf.data.Dataset hoặc generator tùy bài toán.
- **Thông điệp:** dùng fit không có nghĩa là đơn giản hóa quá mức; đây là workflow thực tế cho rất nhiều dự án. Vẫn đủ linh hoạt cho custom loss, metric và callback.

compile() in practice compile() trong thực hành

- **Optimizer** quyết định cách cập nhật trọng số
- **Loss** là đại lượng model cố gắng tối thiểu hóa
- **Metric** giúp con người theo dõi chất lượng dễ hiểu hơn
- Bài toán nhiều output có thể có nhiều loss khác nhau
- Có thể gán trọng số loss để cân bằng các mục tiêu

Writing your own metrics

Tự viết metric

- Metric theo dõi chất lượng nhưng không nhất thiết là loss
- Custom metric kế thừa **keras.metrics.Metric**
- State được tạo bằng **add_weight()** và cập nhật theo từng batch bằng **update_state()**
- **result** trả về giá trị hiện tại của metric
- **reset_state** làm sạch metric trước epoch hoặc evaluation mới

Callbacks during training

Callback trong quá trình huấn luyện

- **Callback** chạy tại các mốc như bắt đầu/kết thúc epoch
- **ModelCheckpoint** lưu model hoặc weight tốt nhất
- **EarlyStopping** dừng train khi validation không cải thiện
- **ReduceLROnPlateau** giảm learning rate khi học bị chững
- Callback giúp điều khiển train mà không viết lại fit
- **Ví dụ thực tế:** train 100 epoch nhưng EarlyStopping có thể dừng ở epoch 18 nếu validation loss không giảm nữa.

Writing custom callbacks

Tự viết callback

- Kế thừa `keras.callbacks.Callback`
- Viết logic trong `on_epoch_end`, `on_batch_end` hoặc hook khác
- Đọc loss và metric hiện tại qua biến logs
- Truy cập model đang train bằng `self.model`
- Phù hợp cho logging, kiểm tra mẫu dự đoán hoặc cảnh báo riêng

TensorBoard monitoring

Theo dõi bằng TensorBoard

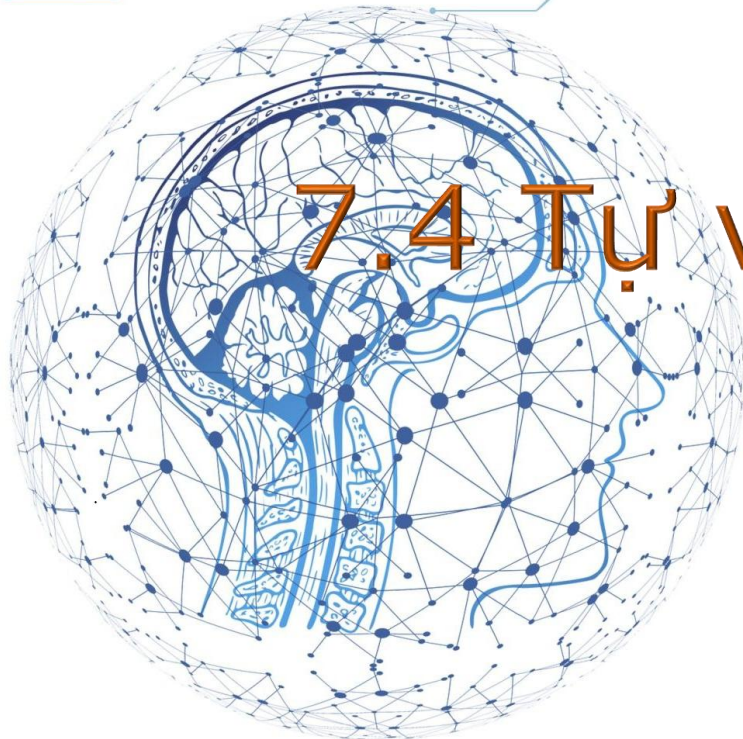
- TensorBoard trực quan hóa loss, metric và nhiều thông tin train
- Keras ghi log thông qua TensorBoard callback
- Có thể so sánh nhiều lần chạy thí nghiệm
- Learning curves giúp nhận diện overfitting hoặc underfitting
- Đây là công cụ quan trọng khi tinh chỉnh model

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU

7.4 Writing your own training and
evaluation loops

7.4 Tự viết vòng lặp huấn luyện và
đánh giá



When to write custom loops Khi nào tự viết vòng lặp?

- Khi thuật toán train không khớp với fit mặc định
- Khi cần nhiều bước cập nhật khác nhau trong một batch
- Khi cần nhiều optimizer, nhiều loss hoặc logic điều kiện phức tạp
- Thường gặp trong research, GAN hoặc thuật toán không chuẩn
- Nếu fit vẫn đủ, nên tiếp tục dùng fit

Training versus inference

Huấn luyện và suy luận

- Một số layer hoạt động khác nhau khi train và khi infer
- Dropout chỉ thả ngẫu nhiên activation trong lúc train
- BatchNormalization dùng thống kê batch khi train
- Khi đánh giá hoặc predict cần gọi model với training=False
- Trong custom loop, quên training mode là lỗi rất phổ biến

Low-level usage of metrics

Dùng metric ở mức thấp

- Tạo metric object trước khi vào vòng lặp
- Sau mỗi batch, gọi `update_state` với `y_true` và `y_pred`
- Dùng result để lấy giá trị metric hiện tại
- Reset metric trước epoch hoặc validation mới
- Quản lý metric state là trách nhiệm của người viết loop

Custom training loop anatomy Cấu trúc vòng lặp train thủ công

- Duyệt từng batch từ Dataset hoặc dữ liệu đã chuẩn bị
- Mở GradientTape để ghi lại phép tính trong forward pass
- Tính prediction và loss cho batch hiện tại
- Tính gradient theo trainable weights của model
- Optimizer áp dụng gradient, sau đó cập nhật metric

Evaluation loop and tf.function

Vòng lặp đánh giá và tf.function

- Evaluation loop không cần GradientTape
- Không cập nhật trọng số trong quá trình đánh giá
- Luôn gọi model với training=False khi evaluate
- tf.function có thể biên dịch step thành graph để chạy nhanh
- Nên debug bằng eager mode trước, tối ưu sau

Leveraging fit with custom train_step

Tận dụng fit với train_step tùy biến

- Subclass keras.Model và override train_step
- Tự định nghĩa logic cho một batch huấn luyện
- Vẫn gọi model.fit như workflow Keras quen thuộc
- Vẫn dùng được callbacks, progress bar, logs và history
- Đây là điểm cân bằng giữa fit mặc định và custom loop hoàn toàn

Chapter Summary

Tổng kết chương

- Keras cung cấp một phổ API từ đơn giản đến tùy biến sâu
- Sequential phù hợp với pipeline tuyến tính, Functional API phù hợp với đa số model thực tế
- Subclassing hữu ích khi kiến trúc cần logic Python động
- Built-in loops nên là lựa chọn mặc định cho huấn luyện và đánh giá
- Custom loop hoặc `train_step` chỉ nên dùng khi bài toán thật sự cần

